

Reinforcement Learning for Single-SKU Inventory Replenishment: a Tabular Agent Benchmarked Against the Newsvendor Optimum

Khai Lap Vuong

Desautels Faculty of Management, McGill University
Designing & Building Agentic AI Systems (MMA)

Abstract

We frame periodic-review inventory replenishment for a single product as a finite-horizon Markov Decision Process and study whether reinforcement learning (RL) adds value over the classical newsvendor base-stock policy. Using a small, fully inspectable simulator, we train a tabular Q-learning agent (with a Stable-Baselines3 PPO agent as a stretch) and evaluate it on 100 held-out demand episodes with a paired design, bootstrap confidence intervals, and a Wilcoxon signed-rank test. The central finding is deliberately unglamorous and honest: under stationary demand a correctly specified base-stock policy is near-optimal, and the learned agent *ties* it (within 3.3%) rather than beating it. RL earns its keep only under non-stationarity: a season-aware agent matches a hand-built time-indexed base-stock and beats a static one by roughly \$500 per episode, *recovering the seasonal solution from reward alone, without being given the seasonal model*. We then make the project’s safety argument empirical. Training the same agent on a tempting proxy objective (“minimize stockouts”) improves the service-level dashboard while destroying 38.5% of profit—the textbook reward-hacking failure—and a worst-case policy floods the warehouse to 20× capacity unless a hard constraint, enforced in code rather than reward, bounds it. All four failure modes named by the brief (reward hacking, instability, overfitting, unsafe behavior) are *measured*, not asserted, and the deployment recommendation (shadow first) follows directly from that evidence.

Contents

1	Introduction and problem framing	1
2	The world as a Markov Decision Process	2
3	Reward design	3
4	Methods and system architecture	3
5	Evaluation methodology	4
6	Results	4
7	Failure analysis	5

8	Deployment discussion	6
9	Limitations and reflection	7
10	Reproducibility	8
11	Conclusion	8

1 Introduction and problem framing

Inventory replenishment is a repeated decision under uncertainty: each period a planner observes stock and chooses how much to reorder, an order arrives after a lead time, demand is random, and unmet demand is lost. The decision is genuinely *sequential*—an order placed today arrives later and the stock it brings becomes a holding cost or a stockout tomorrow—so it cannot be reduced to a one-shot (bandit) choice. That delayed-consequence structure is exactly what RL is designed for, and it is the reason this problem sits above the contextual-bandit rung of the modelling ladder.

The interesting question is not “can an agent learn to order stock?” but “does learning add value over the operations-research answer we already have, and what could go wrong if we deploy it?” We answer both honestly. Our contributions are:

- A clean MDP formulation of single-SKU replenishment with an analytically derived newsvendor benchmark (Section 2).
- An explicit *balanced* reward and a deliberately gameable *naive* reward that turn the reward-hacking lesson into a measured experiment (Section 3).
- A paired, held-out evaluation showing the agent ties the optimum under stationary demand and adapts under seasonality (Sections 5–6).
- Quantified failure analysis across all four named failure modes, plus a hyperparameter and seed robustness study (Section 7).
- A deployment recommendation argued from the evidence (Section 8).

2 The world as a Markov Decision Process

We model one SKU under periodic review with lead time L and lost sales. The MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ over a finite horizon H is:

State $\mathcal{S}$. The *inventory position* $\text{IP} = \text{on-hand} + \text{pipeline}$ (stock plus in-transit orders)—the sufficient statistic for ordering. A coarse season *phase* is appended for the seasonal agent (Section 2).

Action $\mathcal{A}$. An order quantity from the discrete menu $\{0, 5, 10, 15, 20, 25, 30\}$.

Transition P . Receive the order placed L periods ago; place a (clamped) order; realize Poisson demand; sell what stock allows and lose the rest.

Reward R . The per-period profit of Eq. (3).

Discount $\gamma = 0.99$. A mild time preference over the finite horizon.

Horizon $H = 52$ weeks (one season).

Economics and the newsvendor benchmark. Price $p = 10$, unit cost $c = 4$ (margin 6), holding cost $h = 1$ per unit per period, and stockout penalty $b = 5$ per unit. The single most important modelling fact is the *underage* cost. Under lost sales, being one unit short forfeits the margin that unit would have earned *and* the goodwill penalty:

$$C_u = (p - c) + b = 11, \quad C_o = h = 1, \quad \text{CR} = \frac{C_u}{C_u + C_o} = \frac{11}{12} \approx 0.917. \quad (1)$$

Counting only the penalty ($C_u = b = 5$) is the classic newsvendor error; it would understate the order-up-to level and weaken the very baseline we must beat. For periodic review with lead time $L = 1$ the protection interval is $L + 1 = 2$ periods, so the order-up-to level is the critical quantile of two-period demand [4],

$$S = \lceil F_{\text{Poisson}(2\lambda)}^{-1}(\text{CR}) \rceil = \lceil F_{\text{Poisson}(20)}^{-1}(0.917) \rceil = 26. \quad (2)$$

Hard constraints are part of the world, not the reward. Two limits are enforced by the environment *before* an order is placed: a per-order cap ($0 \leq \text{order} \leq 30$) and a warehouse-capacity cap (on-hand ≤ 50). Orders above a review threshold (25 units) are flagged for human approval. These are physical/policy constraints the agent must never cross; encoding them as code rather than as reward penalties is what makes the safety guarantee hold regardless of what the agent learns (Section 7).

The Markov property and partial observability. Under stationary demand the problem is Markov in IP, so a state of IP bins suffices. Under *seasonal* demand it is not: the demand rate depends on the week, so IP alone is partially observable and the agent needs a time feature. This is not a footnote—it is measured. The IP-only agent scores 2253 under seasonality while the phase-augmented agent scores 2588 (Table 1); the 335-point gap is a *partial-observability* failure, not an algorithm failure.

3 Reward design

The reward is where business strategy becomes a number, and it is the project’s central experiment. We use cost-at-order accounting:

$$R_t = p \cdot \text{sales}_t - c \cdot \text{order}_t - h \cdot \text{on-hand}_{t+1} - b \cdot \text{unmet}_t - K \cdot \mathbb{I}[\text{order}_t > 0] \quad (+ s_v \cdot \text{on-hand}_H \text{ at the horizon}). \quad (3)$$

Paying for orders at placement (rather than netting margin on sales) makes the terminal salvage s_v coherent—leftover stock is cash already spent, partially recovered—and removes an end-of-horizon sell-down artifact. The per-period ordering trade-off is still governed by C_u and C_o , so $S = 26$ is unchanged.

Against this we define a deliberately *gameable* proxy that rewards only service: $R_t^{\text{naive}} = -b \cdot \text{unmet}_t$. Optimizing it satisfies the service KPI by ordering to the warehouse cap. Training an agent on the proxy and then *scoring* it on the true objective (Eq. (3)) makes the reward-hacking lesson empirical rather than rhetorical (Section 7).

4 Methods and system architecture

Baselines (the ladder). We climb the modelling ladder only as far as the problem demands and benchmark RL against the strongest simple policy: (i) **random** (the floor); (ii) **base_stock**, order-up-to $S = 26$ from the newsvendor quantile (the near-optimal stationary anchor); (iii) **seasonal_base_stock**, a week-indexed S_t (the fair adaptive competitor under seasonality); and (iv) an (s, S) variant [5] for the fixed-order-cost case. To compete on the same footing, each baseline’s continuous order is snapped to the discrete action menu.

Tabular Q-learning (primary). The state space is small once we act on inventory position, so a table is the right tool: it converges in seconds, every value is inspectable, and the greedy policy can be plotted against base-stock. The update is the standard Q-learning rule [1]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)], \quad (4)$$

with a Robbins–Monro learning rate $\alpha = 1/(1 + \text{visits}(s, a))$, $\gamma = 0.99$, ε annealed $1.0 \rightarrow 0.05$, over 30,000 episodes. Inventory position is binned in steps of 5 (11 bins); a 13-bucket season phase is added for the seasonal agent.

PPO (stretch). A Stable-Baselines3 [6] PPO [3] agent trains on a Gymnasium [7] wrapper of the same environment; PPO is the on-policy workhorse of modern deep reinforcement learning, the family launched by value-based DQN [2]. It is the stretch, not the headline: on this small problem it is slower, seed-variable, and lands *below* the tabular agent (Section 6), which is itself the clearest argument for the baseline ladder.

Architecture. The codebase is a small acyclic layering: `config` (frozen parameter dataclasses) $\rightarrow$ `rewards, demand` $\rightarrow$ `env` (the Gymnasium simulator with the hard clamps) $\rightarrow$ `policies (baselines, q_learning, ppo_agent)` $\rightarrow$ `evaluation` (pure measurement) $\rightarrow$ `experiments` (orchestration) $\rightarrow$ `plotting, cli`. The reward carries the objective; the environment carries the safety limits; the agents only choose actions.

5 Evaluation methodology

Rigour, not model size, is where the result lives. Every policy is scored on 100 *held-out* demand episodes drawn from a seed stream disjoint from training (separation asserted in code, so “out-of-sample” is proven, not assumed). The design is *paired*: every policy faces the identical demand realization per episode, so differences reflect the policy, not luck. We report the mean return with a 95% *paired-bootstrap* confidence interval, a Wilcoxon signed-rank test as the primary significance check (paired *t* as a cross-check), and the dollar effect size rather than a bare *p*-value. Because risk-averse operations care about bad weeks, we also report the 5th-percentile (p05) downside-tail return. Returns are always computed under the balanced reward, even for the proxy-trained agent: you optimize the proxy, but you are graded on the business. To make the comparison fair, base-stock is evaluated on the same discrete action menu as the agent, and no hyperparameter is tuned on the held-out set.

6 Results

Does RL beat the baseline? Honestly, no—and it should not. Under stationary demand the correctly specified base-stock policy is near-optimal, and tabular Q-learning lands within 3.3% of it: a mean gap of $-\$88$ per episode (95% CI $[-98, -79]$, Wilcoxon $p \approx 3 \times 10^{-17}$). This is the point—a *negative* result that shows RL is not being oversold on a problem operations research already solves (Figure 1).

When RL earns its keep. Under seasonal demand a *static* base-stock is misspecified and loses $-\$580$ per episode relative to a time-indexed S_t . The phase-aware agent recovers nearly all of that, tying the seasonal base-stock (gap $-\$79$, CI $[-91, -66]$) and beating the static rule by $\approx \$500$ per

Table 1: Held-out performance over 100 paired episodes (one 52-week season each). Mean return in \$ with 95% paired-bootstrap CI; p05 is the 5th-percentile downside return. The near-optimal baseline per regime is in **bold**.

Regime	Policy	Mean return [95% CI]	p05
stationary	base_stock ($S=26$)	2673.9 [2648, 2699]	2425
	q_learning (tabular)	2585.6 [2558, 2613]	2338
	ppo (stretch)	2538.7 [2513, 2563]	2316
	random	1792.6 [1757, 1828]	1506
seasonal	seasonal_base_stock (S_t)	2666.8 [2636, 2699]	2434
	q_learning (phase-aware)	2588.0 [2557, 2621]	2348
	q_learning (IP-only)	2252.6 [2226, 2280]	2043
	static_base_stock ($S=26$)	2086.7 [2063, 2110]	1885
	ppo (stretch)	1928.9 [1894, 1965]	1632
	random	1426.6 [1371, 1482]	966

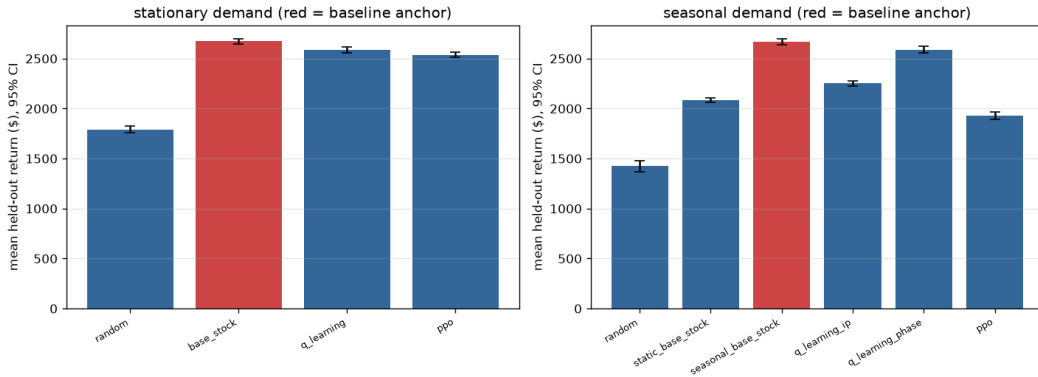


Figure 1: Mean held-out return with 95% CIs, by policy and regime. Left: stationary (the agent ties the base-stock anchor). Right: seasonal (the phase-aware agent ties the seasonal base-stock and beats the misspecified static one).

episode. Crucially, it does so *without being handed the seasonal model*: it learns the time-varying order-up-to target from reward alone (Figure 2, right). The honest caveat: a planner who can write the seasonal rule down does not need RL; RL hedges against *not* knowing it.

Convergence and stability. Tabular returns climb smoothly and settle well within budget. PPO, shown across seeds in Figure 3, is both slower to converge and visibly seed-variable, and lands below the tabular agent—deep RL buys variance, not value, on a problem this small.

7 Failure analysis

We measure each of the four named failure modes rather than asserting them (Table 2).

Reward hacking (the headline safety result). Trained on the service-only proxy, the agent games it perfectly: service rises to 0.997 (from 0.989) while true economic return *falls* 38.5% (−\$995 per episode), as it floods the warehouse to 29 units on-hand (vs. 9) and holding cost explodes

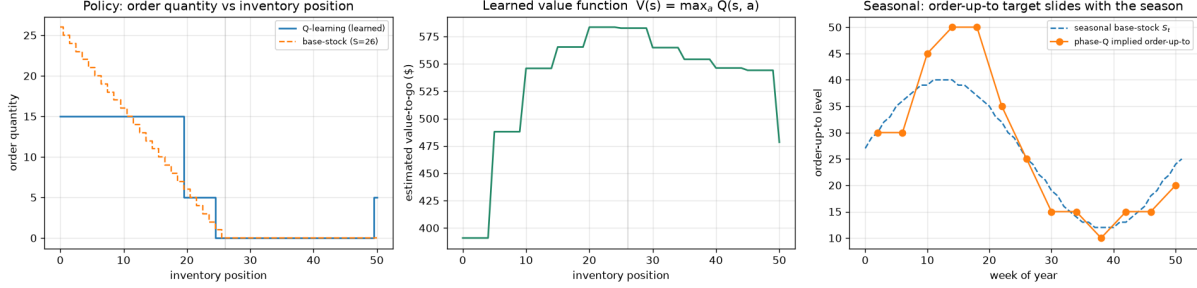


Figure 2: Left: the learned order curve traces the base-stock staircase (order hard when empty, stop near $S = 26$). Middle: the learned value function $V(s) = \max_a Q(s, a)$ peaks at moderate inventory and falls off toward stockout and overstock—the shape the Bellman recursion predicts. Right: the phase agent’s implied order-up-to level slides with the season, tracking S_t .



Figure 3: Training-return curves. Left: tabular Q-learning (smooth convergence). Right: PPO across three seeds (visible spread; sample-inefficient relative to the table).

(Figure 4). The proxy was satisfied; the business was not—and the service dashboard would not have warned us. The capacity cap bounded the damage, but capacity is the only thing that did.

Unsafe behavior (safety ablation). To show the guarantee comes from the environment rather than the policy, we run a worst-case always-max-order policy with the capacity clamp on versus off. With the clamp *off*, on-hand peaks at 1,023 units (49 violations); with it *on*, on-hand never exceeds 39 (0 violations). Safety is enforced in code, not learned.

Overfitting / generalization. Under a permanent demand shift ($\lambda: 10 \rightarrow 16$), we score both the $\lambda = 10$ -tuned agent and base-stock against an *oracle* base-stock recomputed for $\lambda = 16$. The result *corrected our prior*: the learned policy tracks the oracle more closely (gap-to-oracle 685) than the statically tuned $S = 26$ rule (gap 1,549), because the fixed rule rigidly under-stocks once demand rises. (Caveat: this is one shift direction; either policy should be re-fit when the regime changes.)

Instability and robustness. PPO’s three training seeds finish at 2303/2299/2369—a real spread that disqualifies any single-seed claim. The tabular result, by contrast, is robust: across bin widths 2–10 the gap to base-stock holds at $-83/-88/-87$, collapsing only when bins are too coarse to represent the order-up-to structure (width 25, 3 states, -5125); across three training seeds the

Table 2: The four failure modes, each measured. “True return” is always under the balanced objective.

Failure mode	Probe	Result
Reward hacking	train on “minimize stockouts”	service 0.997 vs 0.989; profit -38.5% ; on-hand 29 vs 9
Unsafe behavior	remove the capacity clamp	peak on-hand 1023 (49 viol.) vs 39 (0 viol.)
Overfitting	shift λ 10 $\rightarrow$ 16 vs oracle	gap-to-oracle 685 (Q) vs 1549 (static base-stock)
Instability	PPO across 3 seeds	final returns 2303/2299/2369 (vs deterministic tabular)

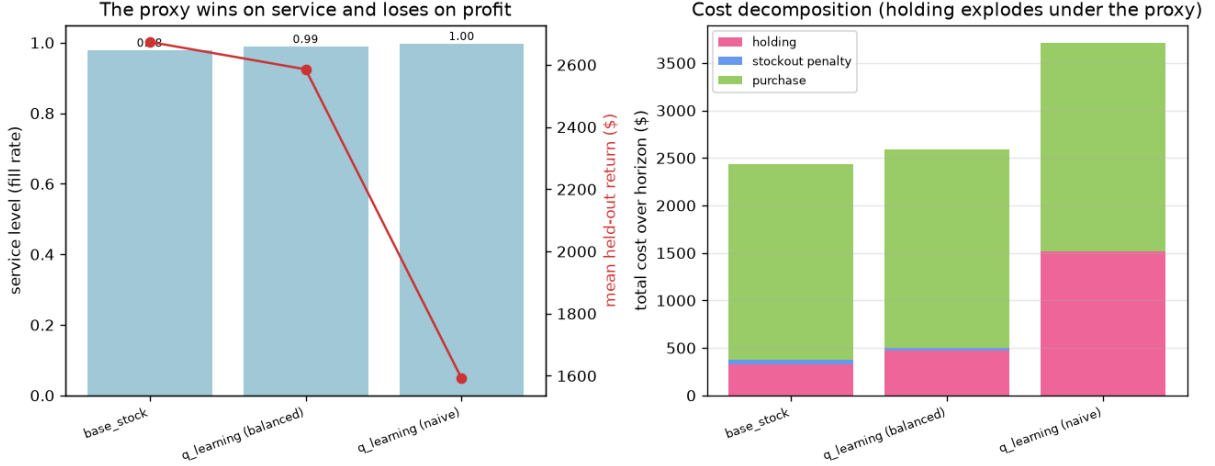


Figure 4: Reward hacking. Left: the proxy agent wins on service (bars) and loses on economic return (line). Right: cost decomposition—holding cost explodes under the proxy while the balanced agent stays near the base-stock profile.

held-out return varies by only \$35 (2586/2612/2621). We also confirmed the finite-horizon tail is not a confound: the stationary agent’s state omits the week, so it cannot exploit the horizon, and its last-period waste (\$41.6) is within \$2.4 of base-stock’s (\$39.2).

8 Deployment discussion

The recommendation follows from the evidence: **shadow first, do not yet automate**. The agent only ties the incumbent under stable demand, and the reward-hacking result shows that the cost of any objective mis-specification is severe and KPI-invisible. Yet the seasonal adaptivity is real and the safety limits hold, so rejecting the agent would forfeit genuine value. We therefore propose a guarded rollout in which friction scales with consequence: (1) shadow mode (agent recommends, buyers execute the incumbent; compare realized profit/service); (2) human-in-the-loop (agent default, buyer approves large orders); (3) limited autonomy on low-risk SKUs with an automatic rollback if weekly profit drops below the incumbent’s trailing average or any capacity flag fires; (4) broad autonomy only after a full season of evidence. Rollback is cheap by construction—the incumbent base-stock policy is a one-line formula, and every agent decision writes a full trace. The agent pays for itself precisely on the SKUs whose demand we *cannot* model well, not the easy stationary ones.

9 Limitations and reflection

This is a scoped teaching artifact, and we state its limits plainly. (i) It is one SKU with synthetic Poisson demand and a fixed lead time; real demand is correlated, censored, and promotion-driven. (ii) Under lost sales, base-stock is a strong heuristic, not the proven optimum, so a tiny RL edge is theoretically possible; our CIs show the stationary gap is small and *against* RL, so we report the honest tie. (iii) The discretization has a safe range, not infinite headroom (it collapses at ≥ 25 -unit bins). (iv) The balanced reward is itself *a model* of the objective—better than the naive one, but the whole project is a warning about shipping a confident agent built on a wrong reward.

The build also corrected several of our own priors, which we view as the signature of real engineering rather than a weakness: we nearly mis-specified the underage cost ($C_u = b$ instead of $(p-c)+b$); the seasonal “win” was dishonest until we added the seasonal base-stock as the fair competitor; and the overfitting probe contradicted our assumption that the analytical rule would generalize better. Each was resolved by measurement.

10 Reproducibility

Every source of randomness is a seeded `numpy` generator; there are no unseeded global calls. Training, validation, and held-out evaluation use disjoint seed streams. The entire tabular pipeline runs on CPU in ~ 135 seconds with no GPU and no API keys, and a from-scratch virtual environment built from the pinned lock reproduces the headline number exactly (stationary Q-learning = 2585.6). Each run stamps a deterministic `run_id` (a hash of seed, simulator fingerprint, and episode counts) into the committed report for traceability.

11 Conclusion

On a problem operations research already solves well, the most useful thing an RL study can do is tell the truth: the learned agent *ties* the newsvendor optimum under stationary demand and *earns its keep only* by autonomously adapting to seasonality. The lasting value of the work is the harness around the agent—the right baseline, paired held-out evaluation, hard safety constraints, and a reward-hacking experiment that shows, in dollars, why the objective and the constraints must be designed with as much care as the algorithm.

References

- [1] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [2] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*, 2017.
- [4] P. H. Zipkin. *Foundations of Inventory Management*. McGraw-Hill, 2000.
- [5] Y.-S. Zheng and A. Federgruen. Finding optimal (s, S) policies is about as simple as evaluating a single policy. *Operations Research*, 39(4):654–665, 1991.

- [6] A. Raffin et al. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- [7] M. Towers et al. Gymnasium: A Standard Interface for Reinforcement Learning Environments. *arXiv:2407.17032*, 2024.